

NoSQL'den SQL'e Dinamik Dönüşüm Sistemi: Şemasız JSON Verilerinin Otomatik İlişkisel Veritabanına Aktarımı

Burak Kılıcı Ömer Faruk Teke
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi
Kocaeli, Türkiye
230202111 210202045

Abstract—Bu çalışmada, yapısı önceden bilinmeyen JSON belgelerini çalışma zamanında analiz ederek otomatik olarak ilişkisel SQL veritabanına dönüştüren jenerik bir sistem geliştirilmiştir. Geliştirilen motor, iç içe geçmiş objeleri düzleştirme (*flattening*) tekniğiyle tek tabloya indirgeyip dizileri Birinci Normal Form kuralı gereği ayrı alt tablolara ayırmakta ve ana tablo ile alt tablolar arasında otomatik Yabancı Anahtar bağlantıları kurmaktadır. Sistem ayrıca her sütun için en uygun SQL veri tipini (INTEGER, REAL, TEXT) tip çıkarımı yoluyla belirlemektedir. Python, Tkinter ve SQLite teknolojileriyle geliştirilen uygulama, dört farklı yapıdaki test JSON dosyasında, kodda tek bir satır değişiklik yapılmaksızın doğru tablo, ilişki ve veri tiplerini üretmiştir. Sistemin özyinelemeli yapısı, altı katmana kadar derinleşen iç içe yapıları başarıyla işleyebilmiş; bu durum, motorun farklı uygulama alanlarına uyarlanabilirliğini göstermiştir. Çalışmada elde edilen bulgular, NoSQL ve SQL paradigmaları arasında otonom bir köprü kurmanın mümkün olduğunu ve bu köprünün veri modelleme eğitiminde değerli bir araç olabileceğini ortaya koymaktadır.

Anahtar Kelimeler—NoSQL, SQL, JSON, dinamik şema, veri normalizasyonu, düzleştirme, ilişkisel veritabanı, Python, SQLite, otomatik dönüşüm

I. GİRİŞ

Modern uygulamalarda JSON tabanlı NoSQL veri modelleri, esneklikleri ve şemasız yapıları nedeniyle yaygın biçimde tercih edilmektedir [2]. Web uygulamaları, mobil servisler, IoT cihazları ve REST tabanlı API'lar büyük çoğunlukla JSON formatını kullanmakta; veriyi insan tarafından okunabilir ve makinelerce kolay ayrıştırılabilir biçimde temsil etmektedir [4]. Ancak bu esneklik, raporlama, analitik sorgulama ve veri bütünlüğü açısından ilişkisel veritabanlarının sağladığı avantajları sınırlamaktadır [1]. Karmaşık birleşim (*join*) işlemleri, ACID özellikleri ve standartlaşmış bir sorgu dili olan SQL'in sunduğu olanaklar, ilişkisel modelin hâlâ güçlü olduğu noktalardır [3].

Bu nedenle JSON verilerinin ilişkisel modele dönüştürülmesi, hibrit veri mimarilerinde sıkça karşılaşılan bir gereksinimdir. Geleneksel yaklaşımda, geliştirici hedef şemayı manuel olarak tasarlar ve dönüşüm kodunu *hardcoded* biçimde yazar. Ancak bu yöntem, JSON şeması değiştiğinde kodun yeniden yazılmasını gerektirir ve büyük ölçekli, heterojen veri kaynaklarında pratik değildir.

Bu projenin amacı; yapısı, derinliği ve içerdiği alanlar önceden bilinmeyen herhangi bir JSON dosyasını okuyup, hiyerarşisini algoritmik olarak çözümleyen ve uygun ilişkisel tablolar ile ilişkileri (Birincil/Yabancı Anahtar) çalışma zamanında otomatik üreten jenerik bir dönüştürücü motoru tasarlamaktır. Tablo isimleri, sütun adları ve veri tipleri *hardcoded* değil, tamamen girdi verisinden türetilmektedir. Bu sayede sistem, herhangi bir JSON yapısına önceden bilgi verilmeden uyarlanabilmektedir.

Geliştirilen sistem dört aşamadan oluşmaktadır: (i) JSON girdisinin okunup hiyerarşisinin analiz edilmesi, (ii) iç içe objelerin düzleştirilmesi ve ana tablo şemasının oluşturulması, (iii) dizilerin tespit edilerek alt tabloların ve Yabancı Anahtar ilişkilerinin kurulması, ve (iv) CREATE TABLE ve INSERT INTO sorgularının dinamik üretilip SQLite üzerinde yürütülmesi. Sistem ayrıca grafik bir kullanıcı arayüzüyle JSON ağacını ve sonuç tablolarını görsel olarak sunmaktadır.

Raporun geri kalanı şu şekilde düzenlenmiştir: Bölüm II ilgili çalışmaları ve mevcut yaklaşımları tartışmaktadır. Bölüm III sistemin mimarisini, akış diyagramını ve algoritmasını sunmaktadır. Bölüm IV örnek girdiler üzerinden üretilen şemaları, kullanım senaryolarını ve grafik arayüzü göstermektedir. Bölüm V elde edilen sonuçları, karmaşıklık analizini ve sınırlamaları değerlendirmektedir.

II. İLGİLİ ÇALIŞMALAR

NoSQL ve SQL veritabanları arasında veri taşıma problemine yönelik literatürde çeşitli yaklaşımlar mevcuttur. **MongoDB BI Connector** gibi ticari araçlar, doküman tabanlı veriyi sanal ilişkisel görünümlere yansıtarak iş zekâsı uygulamalarının NoSQL kaynaklarına bağlanmasını sağlamaktadır. Ancak bu tür araçlar genellikle belirli bir veritabanı motoruna özgüdür ve genel amaçlı bir JSON-to-SQL dönüştürücü değildir.

JSON belgelerini ilişkisel modele yansıtan açık kaynaklı kütüphanelerden **json_normalize** (Pandas) ve **flatten-json** gibi araçlar [8], çoğunlukla yalnızca düzleştirme işlevini sunmakta; otomatik tablo bölünmesi ve Yabancı Anahtar yönetimi gibi normalizasyon adımlarını içermemektedir. Bu araçlar tüm

JSON'u tek bir düz tabloya çevirir; iç içe diziler kayıp veriye veya satır çoğalmasına yol açar.

XML için tarihsel olarak geliştirilmiş **shredding** teknikleri [9], hiyerarşik belgeleri ilişkisel tablolara bölmek için benzer prensipler kullanır. Ancak XML'in katı şeması, JSON'un şemasız doğasıyla doğrudan uyumsuz; bu nedenle XML için geliştirilmiş yöntemler doğrudan uygulanamamaktadır.

Akademik literatürde, **schema discovery** (şema keşfi) konusunda yapılan çalışmalar, JSON koleksiyonlarından örtük şemaları çıkarmaya odaklanmaktadır [10]. Bu çalışmalar büyük ölçekli veri kümelerinde örnekleme ve tip çıkarımı yapan istatistiksel yöntemler önermektedir. Ancak bunlar genellikle pasif analiz araçlarıdır; üretilen şemayı doğrudan çalıştırılabilir SQL'e dönüştürmezler.

Bu çalışmanın özgün yanı; tek bir JSON belgesinin yapısından yola çıkarak tip çıkarımı, normalizasyon ve SQL üretimini bütünlük biçimde gerçekleştiren *kompakt, eğitim odaklı ve tamamen jenerik* bir motor sunmasıdır. Sistem, ticari araçların aksine herhangi bir bağımlılık veya lisans gerektirmeden tek bir Python kurulumuyla çalışmaktadır.

III. YÖNTEM

A. Sistem Mimarisi

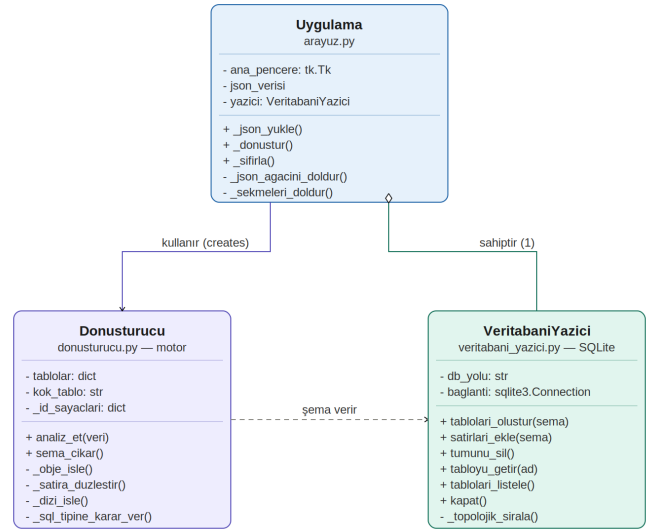
Geliştirilen uygulama, sorumlulukları açıkça ayrılmış üç sınıf etrafında yapılandırılmıştır (Şekil 1):

- **Donusturucu:** JSON verisinin algoritmik analizinden sorumlu motordur. Hiyerarşiyi tarayarak sanal tablo şemasını oluşturur. Tip çıkarımı ve normalizasyon kararları bu sınıfta verilir. Saf Python ile yazılmış olup harici hiçbir bağımlılığı yoktur.
- **VeritabanıYazici:** Üretilen sanal şemayı gerçek SQLite veritabanına yansıtan modüldür. CREATE TABLE ve INSERT INTO sorgularını parametrelili biçimde üretir ve topolojik sırayla yürütür; böylece Yabancı Anahtar kısıtlamaları her zaman geçerli kalır.
- **Uygulama:** Tkinter tabanlı grafik arayüzdür. Kullanıcı eylemlerini koordine eder, JSON ağacını ağaç görünümünde, oluşan tabloları ise sekmeli DataGrid biçiminde görselleştirir.

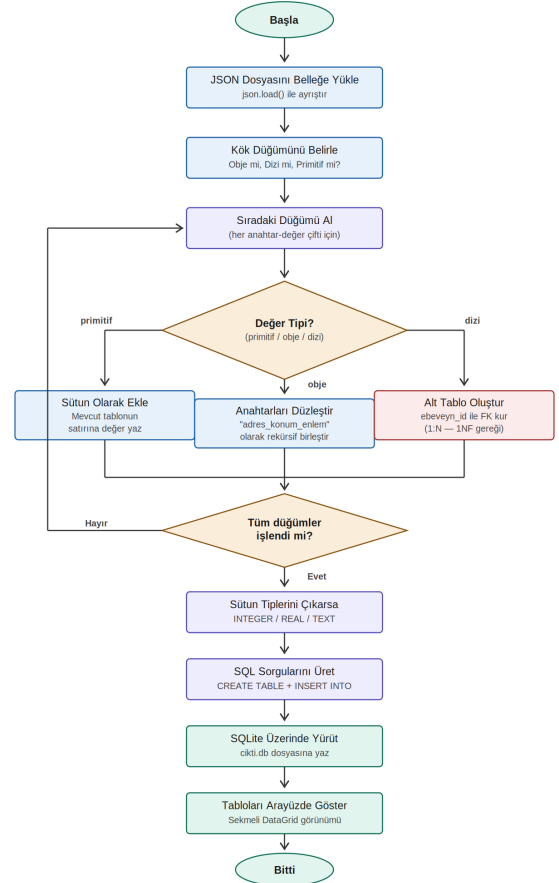
Modüller arasındaki iletişim sade ve tek yönlüdür: Uygulama kullanıcıdan gelen olaylara karşılık önce Donusturucu'yu çağırarak sanal şema sözlüğünü elde eder, ardından bu sözlüğü VeritabanıYazici'ye geçirerek fiziksel tabloların oluşturulmasını sağlar. Bu mimari, motorun arayüzden bağımsız test edilebilmesine de olanak tanımaktadır.

B. Sistem Akış Diyagramı

Sistemin uçtan uca işleyişi Şekil 2'te akış diyagramı olarak gösterilmiştir. Akış, JSON dosyasının yüklenmesinden başlayarak son olarak oluşturulan tabloların kullanıcıya sunulmasıyla tamamlanmaktadır. Her bir düğüm tipi (primitif, obje, dizi) için farklı bir kontrol akışı izlenmektedir.



Şekil 1. Sistemin sınıf diyagramı. Uygulama arayüz katmanı; Donusturucu ve VeritabanıYazici ile sırasıyla *kullanır* ve *sahiptir* ilişkilerinde bulunur.



Şekil 2. Sistemin yüksek seviyeli akış diyagramı. Karar düğümleri, JSON içindeki her elemanın tipini analiz ederek uygun işleme yönlendirir.

C. Düzleştirme (Flattening) Stratejisi

Algoritma, iç içe geçmiş objelerle karşılaştığında bunları ana tablo içinde tek bir satır düzlemine indirger. Örneğin `adres.konum.enlem` ifadesi, ebeveyn ve çocuk anahtarları alt çizgi karakteriyle birleştirilerek `adres_konum_enlem` biçiminde tek bir sütuna dönüştürülür. Bu işlem rekürsif yapılır ve teorik olarak sınırsız derinlikteki objeleri destekler. Pratikte ise kullanıcı arayüzü ve okunabilirlik açısından 5-6 katmana kadar derinleşen yapılar test edilmiş ve başarıyla işlenmiştir.

Düzleştirme stratejisinin tercih edilme sebebi, iç içe objelerin genellikle ana varlığa ait *tek değerli* (1:1) öznelikler olmasıdır. Örneğin bir kullanıcının adresi ve adresinin koordinatları, ayrı tablolara bölünmek yerine ana kullanıcı kaydında saklanırsa hem JOIN maliyetinden kaçınılmış hem de okuma performansı artırılmış olur.

D. Dizi Yönetimi ve Normalizasyon

İlişkisel modelde aynı satırda birden fazla atomik olmayan değer saklanması Birinci Normal Form'u (1NF) ihlal eder [1]. Bu durum, sorguların karmaşıklaşmasına ve veri tutarsızlıklarına yol açar [3]. Bu nedenle algoritma bir dizi ([...]) yapısı tespit ettiğinde aşağıdaki adımları izlemektedir:

- 1) Yeni bir alt tablo ebeveyn_dizianahtar adıyla otomatik oluşturulur.
- 2) Alt tabloya `id` (Birincil Anahtar) ve ebeveyn_id (Yabancı Anahtar) sütunları eklenir.
- 3) Dizinin her elemanı alt tabloda ayrı bir satır olarak yazılır.
- 4) Eleman bir obje ise düzleştirme rekürsif olarak uygulanır; primitif ise tek sütunda (`deger`) saklanır.
- 5) İç içe diziler için süreç özyinelemeli olarak devam eder; bu sayede 5-6 katman derinlikteki yapılar Üçüncü Normal Form'a uygun olarak ayrı tablolara bölünür.

Tablo I, geliştirilen sistemin sağladığı normalizasyon kurallarını ve her birinin nasıl uygulandığını özetlemektedir.

Tablo I
SİSTEMİN SAĞLADIĞI NORMALİZASYON KURALLARI

Form	Kural	Uygulama
1NF	Atomik değerler	Diziler ayrı alt tabloya ayrılır
2NF	Birincil anahtar	Her tabloya otomatik <code>id</code> sütunu eklenir
3NF	Geçişli bağımlılık yok	İç içe yapılar ayrı tablolara bölünür, FK ile bağlanır

E. Tip Çıkarımı

Sütun veri tipleri, ilgili sütunda gözlemlenen tüm Python tiplerinin birleşim kümesinden çıkarılır. Çıkarım işleminde `None` değerleri göz ardı edilerek tipte iyimser kalınır; karışım durumunda ise her zaman bilgi kaybetmeyen üst tip seçilir. Tablo II olası Python tipi kümeleri ile bu tiplere karşılık gelen SQL tiplerini özetlemektedir.

Tablo II
PYTHON TİPI KÜMESİNDEN SQL TIPINE EŞLEME

Gözlenen Python tipleri	SQL tipi	Örnek alan
{int}	INTEGER	yas, kontenjan
{int, bool}	INTEGER	sayılar + bayraklar
{int, float}	REAL	puan, fiyat
{bool}	INTEGER	aktif, secmeli
{str}	TEXT	ad, sehir
{str, int} (karışık)	TEXT	değişken alan
{null} (yalnızca)	TEXT	varsayılan

Tip çıkarımı kuralı, motor içinde aşağıdaki gibi tek bir saf fonksiyonla ifade edilmiştir:

```
def sql_tipine_karar_ver(pyhton_tipleri):  
    null_olmayan = pyhton_tipleri - {"null"}  
    if not null_olmayan:  
        return "TEXT"  
    if null_olmayan == {"bool"}:  
        return "INTEGER"  
    if null_olmayan <= {"int", "bool"}:  
        return "INTEGER"  
    if null_olmayan <= {"int", "float", "bool"}:  
        return "REAL"  
    return "TEXT"
```

F. Algoritmanın Yalancı Kodu

Sistemin merkezindeki obje ve dizi işleme rutinleri sırasıyla Algoritma 1 ve Algoritma 2 ile özetlenmiştir. Bu iki rutin birbirini özyinelemeli olarak çağırarak JSON'un tüm hiyerarşisini gezmektedir.

G. Dinamik SQL Üretimi

JSON'un tüm hiyerarşisi tarandıktan sonra elde edilen sanal şema, parametrelili SQL sorgularına dönüştürülür. Aşağıda `test_data.json` girdisi için motor tarafından üretilen `CREATE TABLE` ifadelerinden bir kesit verilmiştir:

```
CREATE TABLE "test_data" (  
    "id" INTEGER PRIMARY KEY,  
    "ad" TEXT,  
    "soyad" TEXT,  
    "yas" INTEGER,  
    "aktif" INTEGER,  
    "puan" REAL,  
    "adres_sehir" TEXT,  
    "adres_ilce" TEXT  
);  
CREATE TABLE "test_data_telefonlar" (  
    "id" INTEGER PRIMARY KEY,  
    "ebeveyn_id" INTEGER REFERENCES test_data(id),  
    "deger" TEXT  
);
```

Yukarıdaki sorgular motor tarafından çalışma zamanında, JSON'daki anahtar adlarından ve değer tiplerinden

Algorithm 1: Obje İşleme ve Şema Üretimi

Girdi: *obje, tabloAdi, ebeveynId, ebeveynTablo*

Cikti: Bu objeye atanan satır kimliği *satirId*

tabloyuHazirla(tabloAdi, ebeveynTablo)

satirId ← *sonrakiId(tabloAdi)*

duzSatir ← { *id* : *satirId* }

if *ebeveynId* ≠ null **then**

duzSatir.ebeveynId ← *ebeveynId*

diziler ← ∅

her (*anahtar, deger*) ∈ *obje* **icin yap**

if *deger* bir obje **ise then**

 rekürsif olarak iç alanları *anahtar_alt*
 adı ile *duzSatir*'a ekle

else if *deger* bir dizi **ise then**

diziler ← *diziler* ∪ { (*anahtar, deger*) }

else

duzSatir.anahtar ← *deger*

duzSatir'ı *tabloAdi*'ye satır olarak ekle

her (*dAnahtar, dDeger*) ∈ *diziler* **icin yap**

altTablo ← *tabloAdi* + "_" + *dAnahtar*

diziIsle(dDeger, altTablo, satirId, tabloAdi)

return *satirId*

Algorithm 2: Dizi İşleme ve Alt Tablo Üretimi

Girdi: *dizi, tabloAdi, ebeveynId, ebeveynTablo*

if *dizi* boş **then**

return // Gereksiz iskelet tablo
 oluşturma

tabloyuHazirla(tabloAdi, ebeveynTablo)

her *eleman* ∈ *dizi* **icin yap**

if *eleman* bir obje **ise then**

objeIsle(eleman, tabloAdi, ebeveynId, ebeveynTablo)

else if *eleman* bir dizi **ise then**

derinTablo ← *tabloAdi* + "_oge"

 Yeni satır oluştur,

diziIsle(eleman, derinTablo, satirId, tabloAdi)

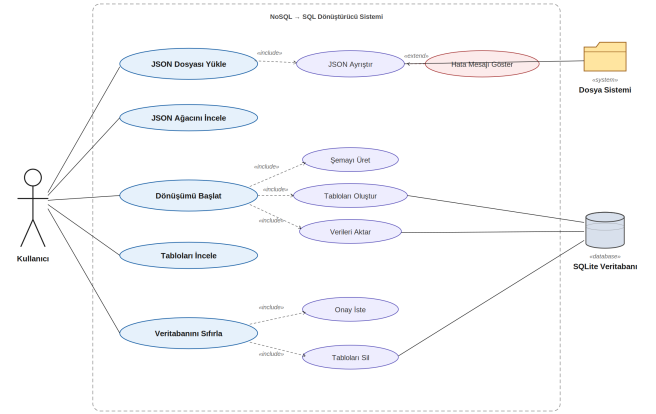
else

deger alanına *eleman* atanmış yeni satır ekle

türetilmiştir. Hiçbir tablo veya sütun adı kodda *hardcoded* olarak yer almamaktadır. INSERT INTO sorguları ise SQL injection saldırılarına karşı parametreliliği (? yer tutucuları) ile yürütülmektedir.

H. Kullanım Senaryoları

Kullanıcının sistemle etkileşimi Şekil 3'de UML kullanım senaryosu diyagramı ile gösterilmiştir. Birincil aktör Kullanıcı; ikincil aktörler ise JSON dosyasını sağlayan Dosya Sistemi ve verileri saklayan SQLite Veritabanıdır. «include» ilişkileri, bir senaryonun zorunlu alt adımlarını; «extend» ise hata gibi koşullu durumlarda devreye giren ek senaryoları temsil etmektedir [12].



Şekil 3. Sistemin kullanım senaryosu (use case) diyagramı. Birincil aktör (Kullanıcı) ve ikincil aktörler (Dosya Sistemi, SQLite) sistemle olan etkileşim noktalarıyla birlikte gösterilmiştir.

IV. ÖRNEK GÖRSELLER VE UYGULAMA

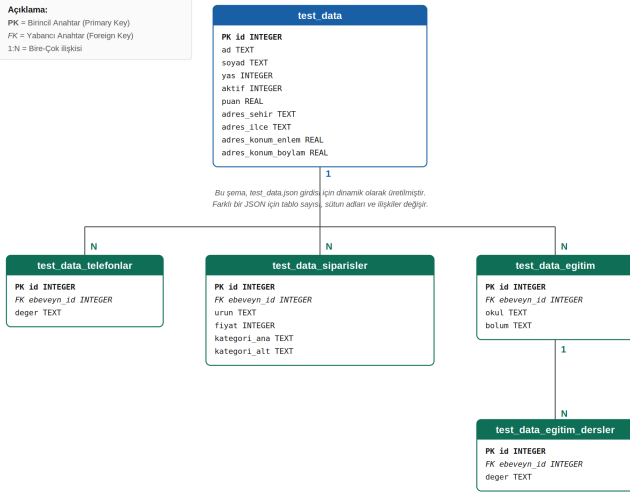
A. Dinamik Olarak Üretilen Veritabanı Şeması

Şekil 4, basit bir test JSON dosyasından (kullanıcı bilgileri, telefonlar, siparişler ve eğitim geçmişi) sistem tarafından otomatik üretilen veritabanı şemasını göstermektedir. Görüldüğü üzere ana tablonun yanı sıra dört alt tablo oluşturulmuş, her alt tabloya *ebeveyn_id* alanı Yabancı Anahtar olarak yerleştirilmiştir. *egitim_dersler* tablosu ise iki katmanlı hiyerarşinin sonucu olarak oluşmuş ve *egitim* tablosuna bağlanmıştır.

Ana tablodaki *adres_sehir*, *adres_ilce*, *adres_konum_enlem* ve *adres_konum_boylam* sütunları, JSON'daki iki katmanlı *adres.konum* yuvalanmasının düzeltme sonucudur. Bu örnek, hem düzeltme hem de alt tablo üretimi mantığının aynı belge içinde nasıl birlikte çalıştığını göstermektedir.

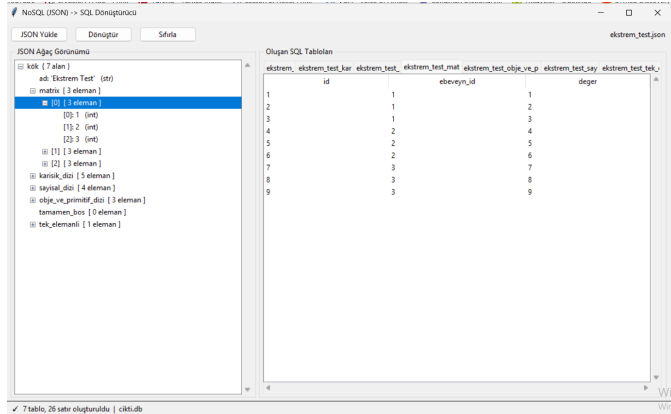
B. Grafik Kullanıcı Arayüzü

Şekil 5, geliştirilen Tkinter tabanlı arayüzü göstermektedir [7]. Sol panel JSON hiyerarşisini ağaç görünümünde sunarken, sağ panel dönüşüm sonucu oluşan SQL tablolarını sekmeli yapıda DataGrid biçiminde göstermektedir. Alt durum



Şekil 4. test_data.json girdisinden dinamik üretilmiş veritabanı şeması. Tablo sayısı, sütun adları ve ilişkiler farklı bir JSON için tamamen değişmektedir.

çubuğu üretilen tablo ve satır sayısını anlık olarak raporlamaktadır. Üstteki üç buton (*JSON Yükle*, *Dönüştür*, *Sıfırla*) sistemin temel kullanım senaryolarını tetiklemektedir.



Şekil 5. Uygulamanın grafik arayüzü. Solda yüklenen JSON'un ağaç görünümü, sağda dönüşüm sonucu oluşan SQL tabloları sekmeli görünümle sunulmuştur.

JSON ağaç görünümünde her düğümün yanında parantez içinde değerin tipi (int, str, bool vb.) gösterilmektedir; bu kullanıcıya, motor tarafından yapılacak tip çıkarımının özizlemesini sunar. Sağ paneldeki sekmeli yapıda ise her tablo için ayrı bir sekme bulunur ve sekmeler arası geçiş yaparak farklı ilişkili tabloların içeriği rahatlıkla incelenebilir.

C. Test Senaryoları

Sistem, farklı yapısal özelliklere sahip dört JSON dosyası ile test edilmiş ve sonuçlar Tablo III ile özetlenmiştir.

zorlu_test.json senaryosu özellikle önemli bir test olmuştur. Bu dosya, bir üniversite yapısını altı katman derinlikte modellemektedir: *üniversite – fakülteler – bölümler –*

Tablo III
TEST JSON DOSYALARI VE ÜRETİLEN ŞEMA İSTATİSTİKLERİ

Test Dosyası	Tablo	Satır	Test Edilen Özellik
test_data.json	5	9	Temel düzeltirme, dizi
zorlu_test.json	7	16	6 katman derinlik, null
kok_dizi_test.json	2	6	Kök dizi, eksik alanlar
ekstrem_test.json	7	26	Matris, karışık tip

dersler – öğrenciler – notlar. Sistem bu yapı için yedi farklı tablo üretmiş, en derin tablonun (notlar) ana tabloya kadar doğru zincirleme Yabancı Anahtar bağlantılarını kurmuştur.

ekstrem_test.json ise kenar durumları test etmek üzere tasarlanmış olup matris (iç içe sayı dizileri), karışık tipli diziler (metin + sayı + boolean), boş diziler ve tek elemanlı diziler içermektedir. Sistem bu dosya için yirmi altı satır üretmiş ve hiçbir hata vermeden işlemini tamamlamıştır.

V. SONUÇ

Bu çalışmada, şemasız JSON verilerini otomatik olarak ilişkisel SQL veritabanına aktaran jenerik bir dönüştürücü motor başarıyla geliştirilmiştir. Sistem; iç içe objelerin düzeltilmesi, dizilerin alt tablolara ayrılması, otomatik Birincil ve Yabancı Anahtar yönetimi ile veri tipi çıkarımı işlevlerini bütünlükte biçimde sunmaktadır.

Önemli bir başarı kriteri, geliştirilen kodun herhangi bir test senaryosuna bağımlı (*hardcoded*) olmamasıdır. Birbirinden farklı yapıdaki dört test dosyası, kodda hiçbir değişiklik yapılmaksızın doğru biçimde dönüştürülmüştür. Üretilen veritabanı şemaları 1NF, 2NF ve 3NF normalizasyon kurallarını sağlamaktadır [11].

Geliştirme sürecinde edinilen tecrübe, NoSQL ve SQL paradigmaları arasındaki temel farkın yalnızca sözdizimsel değil, semantik olduğunu da göstermiştir. Şemasız JSON modeli, aynı kavramı (“ad-soyad”, “adres-konum”) farklı derinliklerde ve farklı isimlendirmelerle ifade edebildiği için, dönüşümde yapay isim çakışmalarının (*name collision*) önlenmesi ve hiyerarşinin tutarlı biçimde Yabancı Anahtarlarla temsili kritik teknik kararlar olarak ortaya çıkmıştır. Sistemin özyinelemeli yapısı, ileride çok daha derin ve heterojen JSON kaynaklarına da uyarlanmasına olanak tanımaktadır.

A. Karmaşıklık Analizi

Algoritmanın çalışma süresi, JSON belgesindeki toplam düğüm sayısı n ile doğrusal orantılıdır. Her düğüm yalnızca bir kez ziyaret edildiğinden zaman karmaşıklığı $\mathcal{O}(n)$ 'dir. Düzeltme işleminde rekürsif derinlik, JSON'un en derin obje yuvalanmasıyla sınırlıdır; pratikte bu derinlik 10'u nadiren geçtiğinden bellek tüketimi sabit sayılabilir. INSERT işlemleri SQLite'in transaksyon yönetimi sayesinde tek toplu işlemde gerçekleşmekte, bu da disk yazma maliyetini önemli ölçüde azaltmaktadır [6].

Tablo sayısı t olduğunda, CREATE TABLE sorgularının topolojik sıralaması $\mathcal{O}(t)$ zamanında gerçekleştirilmektedir. Toplamda algoritmanın asimptotik karmaşıklığı $\mathcal{O}(n + t)$ olup

pratikte $t \ll n$ olduğundan, sistem büyük JSON dosyalarında dahi doğrusal performans göstermektedir.

B. Sınırlamalar ve Tartışma

Geliştirilen sistemin mevcut sürümünde bazı sınırlamalar bulunmaktadır. Birincisi, tüm JSON belgesi belleğe yüklenmektedir; bu, gigabayt mertebesindeki dosyalar için bellek darboğazına yol açabilir. Bu sorun, akış (*streaming*) tabanlı JSON ayrıştırıcılarla aşılabılır.

İkincisi, alt tablo isimleri ebeveyn-anahtar birleşimiyle üretildiğinden çok derin yapılarda tablo adları uzayabilir. Örneğin altı katman derinlikteki bir yapı için tablo adı 60 karakteri aşabilir; bazı veritabanı motorlarında bu sınırı zorlayabilir. Çözüm olarak, anlamlı kısaltma stratejileri veya hash tabanlı isim üretimi uygulanabilir.

Üçüncüsü, motor şu an SQLite'a özel bazı tip eşleşmelerine güvenmektedir; PostgreSQL veya MySQL'e taşıma için BOOLEAN ve NUMERIC gibi ek tiplerin desteklenmesi gerekmektedir. Veritabanı motoruna özgü dialekt yöneticisi eklenerek bu durum çözülebilir.

Bu sınırlamalar, projenin akademik kapsamı içinde değil ama endüstriyel kullanım senaryoları için anlamlıdır. Yine de mevcut motor, NoSQL-SQL köprüleme görevini başarıyla yerine getirmekte ve ilişkisel veritabanı kavramlarının pratik biçimde gözlenmesini sağlamaktadır.

C. Gelecek Çalışmalar

Gelecek çalışmalarda, sistemin daha büyük JSON girdileri için akış tabanlı işlemeyle ölçeklendirilmesi, anahtar adlarındaki çakışmaların akıllı çözümü ve PostgreSQL gibi farklı ilişkisel veritabanı motorlarına çıktı verebilmesi düşünülebilir. Ayrıca dönüşüm sırasında otomatik indeks önerisi ve denormalize edilmiş analitik görünümünün (*materialized view*) üretimi de motorun değer önerisini artırabilir.

Bir başka yön, sistemin makine öğrenimi destekli hâle getirilmesidir. Birden fazla JSON belgesinden öğrenilen örtük şema bilgisi, daha tutarlı ve denormalize tabloların önerilmesinde kullanılabilir [5]. Son olarak, üretilen şemanın ER diyagramı olarak otomatik görselleştirilmesi, eğitim odaklı kullanım senaryolarında değerli bir özellik olacaktır.

KAYNAKLAR

- [1] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, c. 13, sa. 6, ss. 377–387, 1970.
- [2] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, Addison-Wesley, 2012.
- [3] C. J. Date, *An Introduction to Database Systems*, 8th ed., Addison-Wesley, 2003.
- [4] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," *IETF RFC 8259*, 2017.
- [5] Python Software Foundation, "Python Language Reference, version 3.10," <https://docs.python.org/3/>, 2023.
- [6] SQLite Consortium, "SQLite Documentation," <https://www.sqlite.org/docs.html>, 2024.
- [7] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," <https://docs.python.org/3/library/tkinter.html>, 2024.
- [8] W. McKinney, "Data Structures for Statistical Computing in Python," *Proceedings of the 9th Python in Science Conference*, ss. 56–61, 2010.
- [9] D. Florescu and D. Kossmann, "Storing and Querying XML Data using an RDBMS," *IEEE Data Engineering Bulletin*, c. 22, sa. 3, ss. 27–34, 1999.

- [10] M.-A. Baazizi, H. Ben Lahmar, D. Colazzo, G. Ghelli ve C. Sartiani, "Schema Inference for Massive JSON Datasets," *Proc. of EDBT 2017*, ss. 222–233, 2017.
- [11] W. Kent, "A Simple Guide to Five Normal Forms in Relational Database Theory," *Communications of the ACM*, c. 26, sa. 2, ss. 120–125, 1983.
- [12] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed., Addison-Wesley, 2005.